

splunk-cim-reference

A community-maintained reference for Splunk CIM (Common Information Model) work. The headline artifact is a flat CSV of every CIM data model, dataset, field, type, prescribed value, constraint, and description — generated directly from Splunk's CIM app JSON. The repo also includes sourcetype classification tooling, a security device taxonomy, and the parsing script used to (re)generate the CSV.

CIM Data Model Field Reference — start here

The core of this repo is a single flat CSV per CIM release that lists every field across every data model and dataset, with types, required/recommended flags, prescribed values, constraints, descriptions, base searches, and calculated-field expressions. It's the source of truth used by everything else in this repo (the `data_models` column of the sourcetype inventory, the CIM Assessment Toolkit, etc.).

File	CIM version	Status
<code>splunk_data_model_objects_fields_850.csv</code>	8.5.0	Preferred — use this for new work
<code>splunk_data_model_objects_fields_640.csv</code>	6.4.0	Retained for environments pinned to CIM 6.x

Both files share the same column schema, so any consumer (search, transform, AI prompt, dashboard) can target either. Default to `_850.csv` unless you specifically need to match an environment that is pinned to CIM 6.x — in which case use `_640.csv` to avoid flagging fields that don't yet exist in that release. Both files are generated by `parse_cim_datamodels.py` (see [Updating the CIM Field Reference](#) below) and can be regenerated for any future CIM version using the same script.

Human-readable companion

For reading and field-by-field reconciliation (rather than machine consumption), a rendered catalog of the preferred 8.5.0 reference is also included:

File	Description
<code>cim_data_model_field_catalog.md</code>	Human-readable Markdown catalog of every CIM 8.5.0 field, organized by model → dataset, with status (<code>Req/Rec/Opt</code>), type, tags, constraints, prescribed values, and descriptions. Derived directly from <code>splunk_data_model_objects_fields_850.csv</code> (1474 fields · 27 models · 139 datasets). Use this to reconcile a generated or third-party add-on field-for-field against what the model actually defines. Note that the <code>Splunk Audit Logs</code> section has been removed.
<code>cim_data_model_field_catalog.pdf</code>	PDF rendering of the same catalog.

Contents

Sourcetype Classification

File	Description
<code>cim_sourcetype_inventory.csv</code>	Master reference of common Splunk sourcetypes classified by vendor, security relevance, scope, applicable CIM data models, and exclusion status. Starting point for CIM normalization projects.
<code>cim_sourcetype_inventory.csv.version.csv</code>	Sidecar version/provenance file for the inventory. Single-row CSV (<code>last_updated</code> , <code>updated_by</code> , <code>note</code> , <code>base_catalog_last_updated</code>) that the CIM Assessment Toolkit reads to report which inventory is loaded and to distinguish the bundled reference inventory from a custom, environment-specific one. See Sourcetype Inventory Version Sidecar .
<code>sourcetype_analysis_prompt.md</code>	Prompt and instructions for using an AI assistant to classify a new list of sourcetypes into the inventory format.
<code>sourcetypes_list.csv</code>	Example input file — a raw list of sourcetypes and event counts from a <code>tstats</code> search. Illustrates the expected input format for the analysis prompt.

Security Classification Taxonomy

File	Description
<code>security_classifications_reference.md</code>	Authoritative reference for IT security device classifications used in CIM normalization. Defines 40 acronyms (EDR, NGFW, IAM, SIEM, etc.) with descriptions, vendor examples, and classification guidelines. Use this as the source of truth for <code>security_classification</code> values in the sourcetype inventory.
<code>security_classifications_reference.pdf</code>	PDF version of the same reference.
<code>Security-Categories-and-Acronyms.md</code>	Quick-reference cheat sheet of security acronyms organized by category. A condensed companion to the full reference above — useful for fast lookups. Note that the full reference is authoritative; a handful of acronyms (FW, NGFW, UTM, SWG, EPP, CTD, ESG, DT, VPN) appear there but not here.
<code>IT-Security-Device-Classifications.md</code>	Broader classification reference covering 15 security device categories with descriptions, vendor examples, applicable Splunk CIM data models, and common sourcetypes for each category. Useful for understanding

File	Description
	which CIM data models a given security device type maps to.

CIM Data Model Parsing Tools

File	Description
<code>parse_cim_datamodels.py</code>	Python script (standard library only) for parsing Splunk's CIM JSON data model files into tabular CSV format. Accepts <code>--models-dir</code> (default <code>./models</code>), <code>--output</code> , <code>--cim-version</code> , and <code>--force</code> . If a <code>Splunk_SA_CIM app.conf</code> is reachable from the models directory, the CIM version is auto-detected and the default output filename becomes <code>splunk_data_model_objects_fields_<version_no_dots>.csv</code> ; otherwise it falls back to the unversioned <code>splunk_data_model_objects_fields.csv</code> . The script refuses to overwrite an existing output file unless <code>--force</code> is passed.
<code>Data_Model_JSON_Parser.ipynb</code>	Jupyter notebook version of the same — useful for interactive exploration and spot-checking.

Note: The parsing tools require access to Splunk's CIM app JSON files, typically found at `$SPLUNK_HOME/etc/apps/Splunk_SA_CIM/default/data/models/`.

Sourcetype Inventory Format

`cim_sourcetype_inventory.csv` uses the following columns:

Column	Description
<code>sourcetype</code>	Splunk sourcetype name
<code>events</code>	Event count (set to 0 in the starter inventory — populate from your environment)
<code>mapped_status</code>	<code>mapped</code> , <code>unmapped</code> , or <code>excluded</code>
<code>scope</code>	<code>security</code> , <code>operational</code> , <code>security,operational</code> , <code>none</code> , or <code>unknown</code>
<code>security_classification</code>	Acronym(s) from <code>security_classifications_reference.md</code> (e.g., <code>EDR</code> , <code>NGFW</code>). <code>N/A</code> for non-security sourcetypes.
<code>security_relevance</code>	<code>high</code> , <code>med</code> , <code>low</code> , or <code>none</code>
<code>data_models</code>	Applicable CIM data models in <code>Model.Dataset</code> format (e.g., <code>Authentication.Authentication</code>). Up to 3, drawn from the CIM field reference CSV (preferably <code>splunk_data_model_objects_fields_850.csv</code> ; <code>_640.csv</code> for CIM 6.x environments).

Column	Description
<code>vendor</code>	Technology vendor
<code>description</code>	Brief description (50 characters max)
<code>exclude</code>	Y or N — controls whether the sourcetype is filtered from unmapped counts in CIM compliance tools
<code>exclude_reason</code>	Reason for exclusion if <code>exclude=Y</code>
<code>reviewed_by</code>	Who classified this entry
<code>reviewed_date</code>	Date reviewed (M/D/YYYY)

Sourcetype Inventory Version Sidecar

`cim_sourcetype_inventory.csv` ships with a sidecar file that records the inventory's provenance: `cim_sourcetype_inventory.csv.version.csv`. The CIM Assessment Toolkit (CAT) reads this sidecar to report which inventory is loaded and to distinguish the bundled reference inventory from a custom, environment-specific one.

Naming convention: the sidecar is the inventory filename with `.version.csv` appended (`cim_sourcetype_inventory.csv` → `cim_sourcetype_inventory.csv.version.csv`). Keep the two files together.

Format: a single data row under a header, with these columns:

Column	Description
<code>last_updated</code>	Date this inventory (bundled or customized) was last updated (YYYY-MM-DD)
<code>updated_by</code>	Email or identity of whoever produced this inventory
<code>note</code>	Free-text note — e.g., that this is the bundled reference inventory, or a custom inventory for a specific environment
<code>base_catalog_last_updated</code>	The <code>last_updated</code> date of the standard bundled catalog this inventory is built on (YYYY-MM-DD). For the bundled reference inventory this equals <code>last_updated</code> . For a customized inventory it stays pinned to the base catalog release the customizations were layered on top of, so operators can tell which upstream catalog version a custom inventory derives from — and detect when a newer base catalog is available to re-merge against.

```
last_updated,updated_by,note,base_catalog_last_updated
"2026-05-17","jim.baxter@machinedatainsights.com","Initial addition of the
cim_sourcetype_inventory.csv.version.csv file","2026-05-17"
```

When delivering a customized `cim_sourcetype_inventory.csv` to CAT — for example, one extended with sourcetypes that are unique to a particular environment (custom apps and the like) or not yet in the standard catalog — supply a matching sidecar whose `note` flags it as a custom inventory and whose `last_updated` / `updated_by` reflect that change. Leave `base_catalog_last_updated` set to the `last_updated` value of the standard catalog the customizations were built on; don't advance it when you edit the custom inventory. This decouples "when was this (custom) inventory last touched" from "which release of the standard catalog it descends from," so when the bundled catalog is updated an operator can compare the two dates and decide whether to re-merge their customizations onto the newer base. CAT surfaces these values so operators can tell at a glance which inventory is in effect and how current its base catalog is. The sidecar uses CSV (rather than JSON) so it can be loaded by CAT alongside the inventory CSV using the same tooling.

Using the Sourcetype Analysis Prompt

To classify a new batch of sourcetypes:

1. Run a `tstats` search in Splunk to get your sourcetype list with event counts:

```
| tstats count WHERE index=* NOT index=_* BY sourcetype
| sort - count
| rename count as events
```

2. Export the results as `sourcetypes_list.csv`
 3. Open an AI assistant session and attach:
 - `sourcetypes_list.csv` (your input)
 - `security_classifications_reference.md` (or `.pdf`)
 - `splunk_data_model_objects_fields_850.csv` (preferred; or `_640.csv` if your environment is on CIM 6.x)
 - `sourcetype_analysis_prompt.md` (as the prompt)
 4. The output is a `cim_sourcetype_inventory_<timestamp>.csv` ready to merge into the master inventory
-

Updating the CIM Field Reference

`splunk_data_model_objects_fields_850.csv` (preferred) was generated from Splunk CIM v8.5.0, and `splunk_data_model_objects_fields_640.csv` from CIM v6.4.0. To regenerate either for a newer CIM version:

1. Locate the CIM app JSON files on your Splunk server:

```
$SPLUNK_HOME/etc/apps/Splunk_SA_CIM/default/data/models/
```

Copy the `.json` files (Alerts.json, Authentication.json, etc.) into a local `./models` folder. If you also keep the app's `default/app.conf` reachable from the models directory (either alongside it or two levels up at `default/app.conf`, as in the original Splunk app layout), the script will auto-detect the CIM version.

2. Run `parse_cim_datamodels.py` (or use the Jupyter notebook) against that directory:

```
python parse_cim_datamodels.py
```

- If a CIM version is detected (e.g., `8.5.0`), the output is written to `./splunk_data_model_objects_fields_850.csv` (dots stripped from the version).
- If no version is detected and `--cim-version` is not supplied, the output falls back to the unversioned `./splunk_data_model_objects_fields.csv`.
- To override or supply the version manually:

```
python parse_cim_datamodels.py --cim-version 8.5.0
```

- To pick an explicit filename:

```
python parse_cim_datamodels.py --output  
./splunk_data_model_objects_fields_850.csv
```

- The script refuses to overwrite an existing output file. Pass `--force` to allow overwrite:

```
python parse_cim_datamodels.py --force
```

Note: Earlier revisions of this script wrote a single fixed filename unconditionally. The current behavior is to auto-detect the version from `app.conf` (or accept `--cim-version`) and embed it in the default filename — which is how `_640.csv` and `_850.csv` were produced. The prior version of the script is preserved under `/archive/parse_cim_datamodels.py`.

3. The output CSV can replace the existing file — update the filename reference in `transforms.conf` in any apps that use it.

Related Projects

- [CIM Assessment Toolkit \(CAT\)](#) — Splunk app that uses this reference data to measure CIM compliance across data models. Includes dashboard, automated report generation, and scheduled email delivery.
- [Splunk CIM Add-on](#) — Official Splunk CIM app containing the data model definitions this reference is derived from.

Contributing

This repository does not accept pull requests. Maintainer bandwidth is limited and we can't take on the review burden.

Suggestions, corrections, additional sourcetypes, or classification updates are still very welcome — please send them via the contact form at machinedatainsights.com and we'll fold appropriate updates into a future release.

License

Reference materials in this repository are released under [Creative Commons Attribution 4.0 International \(CC BY 4.0\)](#). Scripts are licensed under [Apache License 2.0](#).

Machine Data Insights Inc. — *"There's Gold In That Data!"*